AUTOMATED PORK RASHER PACKAGING DEFECT DETECTION

Computer Vision for Automated Quality Inspection

MMA3001 – Numerical Methods and Machine Learning

Individual Project Report

Student: Sameer Simon 32169604

Repository: https://github.com/Sameer462742/MMA_3001_Project.git

Abstract

This project investigates computer-vision methods for detecting and localising packaging defects in pork rasher product images. The system uses a Roboflow-hosted RF-DETR workflow for inference and YOLOv8n for local training and augmentation experiments. A Python-based Streamlit application supports image upload, annotated prediction visualisation, inspection summaries and CSV/ZIP export. On the reported evaluation runs, the corrected-augmentation YOLOv8n model achieved a precision of 0.5204, recall of 0.8009, mAP@50 of 0.5987 and mAP@50–95 of 0.2102. The RF-DETR workflow achieved a precision of 0.7733, recall of 0.6374, F1-score of 0.6988, mAP@50 of 0.5487 and mAP@50–95 of 0.2363. These results suggest a trade-off between recall and precision. However, differences in evaluation implementations mean that the comparison should be treated as provisional until both models are evaluated using consistent settings. Dataset class imbalance and the absence of labelled wrinkle instances in the test split further limit the conclusions. The project demonstrates a prototype inspection workflow, but additional validation is required before production deployment.

1. Engineering Problem and Objectives

1.1 Engineering context

Packaging inspection is an important quality-control activity in food production. Manual inspection can be repetitive and can vary with operator judgement, inspection conditions and product throughput. A computer-vision system can assist an operator by highlighting regions that may contain packaging or product-presentation defects. The system developed here is a prototype decision-support tool; it is not a certified food-safety system and should not independently authorise products for release.

1.2 Problem definition

The input is an RGB image of a pork rasher product and its packaging. The output is a set of predicted defect instances, each represented by a class label, confidence score and rectangular bounding box. If no prediction passes the selected confidence threshold, the system reports that no defect was detected above that threshold; this does not establish that the image is defect-free.

1.3 Objectives

- Develop a reproducible object-detection workflow for five packaging/product-presentation defect classes.
- Run hosted RF-DETR inference and visualise detections in a usable Python application.
- Use YOLOv8n experiments as a baseline and investigate augmentation as a model-improvement strategy.
- Evaluate performance using ground-truth-based detection metrics, per-class analysis and runtime measurements.
- Document limitations, reproducibility, testing and responsible use of AI tools.

1.4 Scope and limitations

The scope is image-based defect detection rather than end-to-end factory automation. The current application relies on network access and a hosted Roboflow workflow. Its reliability is affected by training-data coverage, annotation consistency, confidence-threshold selection, workflow availability and whether prediction coordinates match the original image dimensions. Conveyor integration, camera calibration, line-speed validation and production safety certification are outside the current scope.

2. Inputs, Outputs and Dataset

2.1 Input and output specification

Element	Specification	Purpose
Input image	JPG, JPEG, PNG, BMP or WEBP	Photograph of a pork rasher product/package.
Prediction class	One of the supported defect labels	Categorises the detected defect.
Confidence	Model score, usually 0–1	Ranks prediction confidence; not a verified probability of correctness.
Bounding box	Centre x/y, width and height in workflow coordinate space	Localises the predicted defect region.
Inspection status	DEFECT_DETECTED / NO_DEFECT_DETECTED / INFERENCE_ERROR	Separates detections, no qualifying detections and failed inspections.
Outputs	Annotated images, CSV report, summary chart and ZIP archive	Supports review and downstream analysis.

2.2 Defect classes

- `loose-meat`
- `packaging-error`
- `twisted-meat`
- `unsealed`
- `wrinkle`

Class coverage is not uniform. Previous dataset inspection found very few training examples for `wrinkle`, and prior validation/test splits had no instances of this class. Consequently, a class-level score for wrinkle cannot be considered reliable unless the evaluation set is corrected to contain representative labelled examples.



Figure 1: Training-set defect class distribution

The training data are strongly imbalanced: packaging-error has substantially more instances than the other classes, while wrinkle has very few. This imbalance can bias the detector towards common classes and makes performance on rare defects less dependable.

2.3 Data quality

Two annotation files were previously identified as potentially malformed or mixing segmentation and detection formats. These should be reviewed, corrected and re-exported before the final training run. The dataset version, split definitions, image counts and per-class instance counts should be recorded in the final report. Any corrected dataset must be the actual dataset used to train the model being reported.

3. Computational Method

3.1 Hosted RF-DETR inference

The deployed inference route calls a Roboflow-hosted workflow through the Inference SDK. For each uploaded image, the application validates the file, submits it to the workflow, extracts the prediction list, filters predictions using a configurable confidence threshold, and draws bounding boxes and labels on a copy of the image. The workflow identifier currently configured in the application is `pork-rasher-errorpackaging-vpork-rasher-error-packaging-rj5da-1-rfdetr-small-t1-logic` in workspace `ece4078h3`. This identifies the deployed service; it does not by itself establish the model's accuracy.

3.2 YOLOv8n baseline and augmentation experiment

YOLOv8n was trained on the exported object-detection dataset and evaluated using precision, recall, mAP@50 and mAP@50–95. An augmented training experiment was used to examine whether additional variation during training could improve generalisation. Comparisons are meaningful only when the dataset split, evaluation configuration and metric definitions are consistent.

3.3 Application architecture

- Interface: Streamlit browser UI for uploading one or more images and selecting a confidence threshold.

- Inference: Roboflow Inference SDK sends images to the hosted workflow.
- Post-processing: predictions are parsed and checked for required fields; qualifying boxes are clipped to image bounds for drawing.
- Reporting: results are summarised and exported as CSV; annotated images are packaged in a ZIP archive.
- Error handling: inference failures are recorded separately from images with no detections above threshold.

A coordinate-system check is required: the current drawing logic assumes workflow bounding-box coordinates refer to the original uploaded image dimensions. If the workflow resizes or crops an image, coordinates may need to be transformed before drawing.

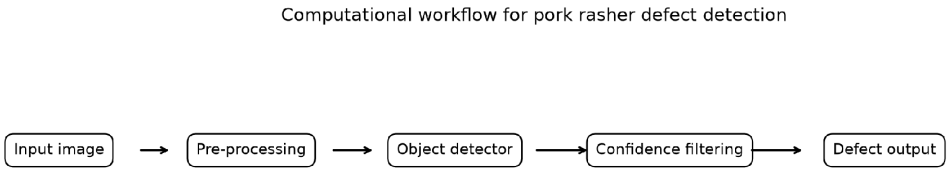


Figure 2: Computational workflow for pork rasher defect detection

The workflow separates image validation, hosted RF-DETR inference, prediction filtering and visualisation, and report export. Inference errors should remain distinct from images for which no prediction exceeds the confidence threshold.

3.4 Key parameters

Parameter	Current configuration / reporting requirement
Default confidence threshold	0.50
Input formats	JPG, JPEG, PNG, BMP, WEBP
Inference backend	Roboflow-hosted RF-DETR workflow
Comparison model	YOLOv8n
Final evaluation split	**Final Evaluation Split:** Roboflow Pork Rasher Error Packaging dataset, Version 1 (V1): 1,955 images in total, comprising 1,764 training images, 115 validation images and 76 test images. The held-out test split contains 76 images.
Runtime environment	Mac OS, Python version 3.13.16, `inference-sdk` 1.7.4, Streamlit 1.65.0, An active internet connection is required to access the hosted inference service.

4. Validation and Results

4.1 Metrics

Precision measures the fraction of predicted detections that match ground truth. Recall measures the fraction of ground-truth objects detected. F1-score summarises the precision–recall balance. mAP@50 uses an IoU threshold of 0.50, while mAP@50–95 averages detection performance over IoU thresholds

from 0.50 to 0.95. Runtime should be measured over a representative batch and reported with the timing scope stated (for example, API round-trip time versus model-only inference).

4.2 Previously recorded YOLOv8n results

Experiment	Precision	Recall	mAP@50	mAP@50–95
Baseline validation	0.584	0.606	0.644	0.337
Augmented validation	0.459	0.737	0.680	0.373
Baseline test (separate run)	0.670	0.560	0.611	0.253

These values were recorded in earlier experiments. The baseline validation, augmented validation and baseline test results are not a single controlled comparison because they refer to different evaluation runs or splits. They should not be combined into one ranking without rerunning evaluation under consistent settings. The augmentation validation run increased recall and mAP while reducing precision, suggesting a trade-off between finding more labelled defects and producing more false positives.

4.3 RF-DETR and YOLOv8n Corrected-Augmentation Results

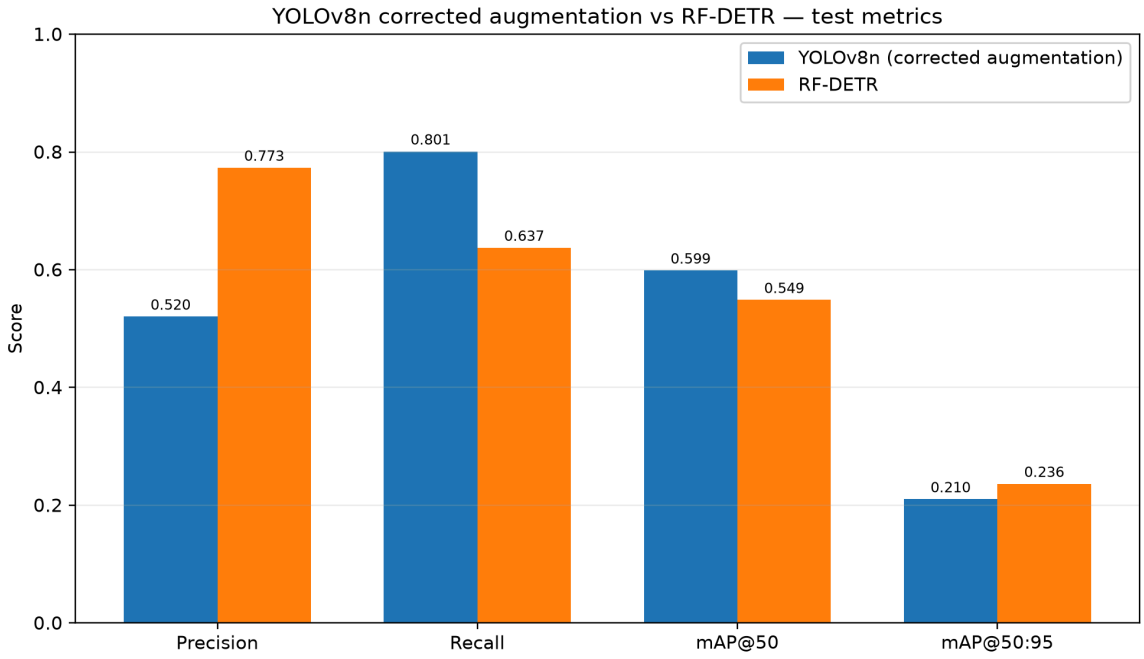


Figure 3: Yolov8n corrected augmentation vs REF-DETR

The hosted RF-DETR workflow was evaluated on 76 images from the Roboflow Pork Rasher Error Packaging Version 1 test split. The test split contained 91 labelled object instances across four classes: loose-meat, packaging-error, twisted-meat and unsealed. The wrinkle class had no labelled instances in the test split, so performance for this class could not be measured.

At a confidence threshold of 0.50 and an IoU matching threshold of 0.50, the custom RF-DETR evaluator reported precision of 0.7733, recall of 0.6374 and F1-score of 0.6988. Its reported mAP@50 was 0.5487 and mAP@50–95 was 0.2363. The corrected-augmentation YOLOv8n test evaluation reported precision of 0.5204, recall of 0.8009, mAP@50 of 0.5987 and mAP@50–95 of 0.2102. Its F1-score, calculated from the reported precision and recall, is approximately 0.6309.

On these reported runs, RF-DETR has higher precision and F1-score, whereas YOLOv8n has higher recall and mAP@50. RF-DETR has a higher mAP@50–95. These differences should be interpreted cautiously because the RF-DETR custom evaluator and the Ultralytics YOLO evaluator may use different matching, aggregation or confidence-handling procedures. A definitive comparison requires both models to be evaluated on the same labelled images with consistent class mappings, bounding-box coordinate conventions, IoU thresholds and metric calculations.

The RF-DETR packaging-error class remains a key weakness: its recall was 0.3793, meaning that many labelled packaging-error instances were missed in this evaluation. The low number of loose-meat instances and the absence of wrinkle instances also limit class-level conclusions.

Measure	YOLOv8n corrected augmentation	RF-DETR workflow	Interpretation / caveat
Test images	76	76	Confirm both used the identical images.
Ground-truth instances	91	91	Four classes represented; wrinkle has zero test instances.
Precision	0.5204	0.7733	RF-DETR higher on reported evaluator.
Recall	0.8009	0.6374	YOLOv8n higher on reported evaluators.
F1-score	0.6309*	0.6988	YOLO F1 calculated from precision and recall; RF-DETR reported by custom evaluator.
mAP@50	0.5987	0.5487	Metric implementation should be aligned before ranking.
mAP@50–95	0.2102	0.2363	Metric implementation should be aligned before ranking.
Mean time per image	33.43 ms (reported)	703.90 ms	Different timing scopes; RF-DETR includes network/API overhead.
Median time per image	Not recorded	653.08 ms	Record YOLO per-image timing for a more useful comparison.
Inference failures	0/76 reported	0/76 reported	Confirm YOLO failure status from its actual evaluation log.

* YOLOv8n F1-score is calculated as 2 × precision × recall / (precision + recall). The model comparison remains provisional until both evaluators are aligned.

Per-class RF-DETR results

Ground-truth counts and metrics from `metrics.json`.

Class	GT instances	Precision	Recall	F1-score	mAP@50 / mAP@50–95
loose-meat	3	1.000	0.667	0.800	0.667 / 0.333
packaging-error	29	0.647	0.379	0.478	0.414 / 0.086
twisted-meat	7	0.500	0.714	0.588	0.357 / 0.163
unsealed	52	0.870	0.769	0.816	0.757 / 0.362
wrinkle	0	—	—	—	—

4.4 Evaluation Procedure and Limitations

The RF-DETR workflow processed all 76 images in the Version 1 test split, and no inference failures were recorded for this run. Prediction records were saved to CSV, while summary metrics and per-class results were saved as JSON files. These outputs provide a reproducible record of the evaluation and support subsequent analysis.

Precision, recall and F1-score were calculated using a confidence threshold of 0.50 and an IoU matching threshold of 0.50. Mean and median request times were recorded to assess the responsiveness of the hosted workflow. The reported latency includes network communication and hosted-service response time.

The results must be interpreted in the context of class imbalance and the evaluation implementation. The test split contained no labelled wrinkle instances, limiting conclusions for that class. The custom RF-DETR evaluation should be checked for bounding-box coordinate consistency and evaluated using the same implementation and settings as YOLOv8n before a definitive ranking is reported.

5. Optimisation and Alternative Solutions

5.1 Baseline comparison

YOLOv8n provides a local baseline with reported validation and test metrics. The hosted RF-DETR workflow provides a second detection approach with a convenient deployment interface. A defensible choice between them should consider accuracy, class-level recall, latency, network dependence, cost/usage limits, model availability, maintainability and ease of reproducibility. Hosted API response time should not be compared directly with local model-only inference time without clarifying the difference.

5.2 Confidence-threshold sensitivity

Evaluate a predefined threshold grid (for example 0.25, 0.35, 0.50, 0.65 and 0.75) on validation data. Record precision, recall, F1 and false-positive/false-negative counts for each threshold. Select an operating point based on the inspection objective and freeze it before evaluating the final test set. If per-class thresholds are considered, ensure that every class has enough representative validation examples.

5.3 Runtime graph caption and explanation

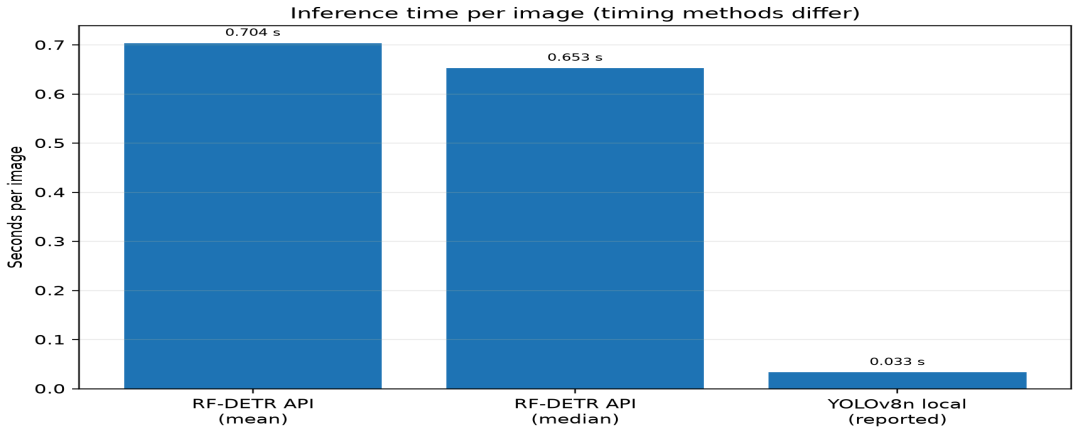


Figure 4. Reported inference time per image for the hosted RF-DETR workflow and local YOLOv8n model.

The mean RF-DETR request time was approximately 0.704 seconds per image and the median was 0.653 seconds. These measurements include network communication with the hosted inference service. The

reported YOLOv8n timing is approximately 33.43 milliseconds per image and represents local inference. Because these timing measurements cover different execution pathways and may have been collected using different procedures, they provide application-level context rather than a controlled speed benchmark. Future testing should measure both systems over the same image batch, with timing boundaries and hardware/network conditions documented.

6. Engineering Interpretation

From an engineering perspective, the key result is not simply whether a model produces bounding boxes, but whether it detects the defect types that matter at an acceptable false-alarm rate and processing time. The low RF-DETR recall for packaging-error highlights the risk of missed defects, while the higher reported YOLOv8n recall may be useful where missing a defect is particularly costly. The final system choice should therefore consider the relative cost of false negatives and false positives, as well as latency, network dependence, maintainability and the reliability of the evaluation.

7. Application Use and Reproducibility

7.1 Running the app

1. Install dependencies using `python -m pip install -r requirements.txt` in the project's active environment.
2. Configure the Roboflow API key securely. The local testing version may use a placeholder constant, but a real key must not be committed to GitHub.
3. Start the interface from the project directory using `python -m streamlit run app.py` (not `python app.py`).
4. Open the local URL printed by Streamlit, usually `http://localhost:8501`.
5. Set a confidence threshold, upload JPG/JPEG/PNG/BMP/WEBP images, and click Run defect detection.
6. Review each annotated image and its status, then download the CSV and annotated-image ZIP.

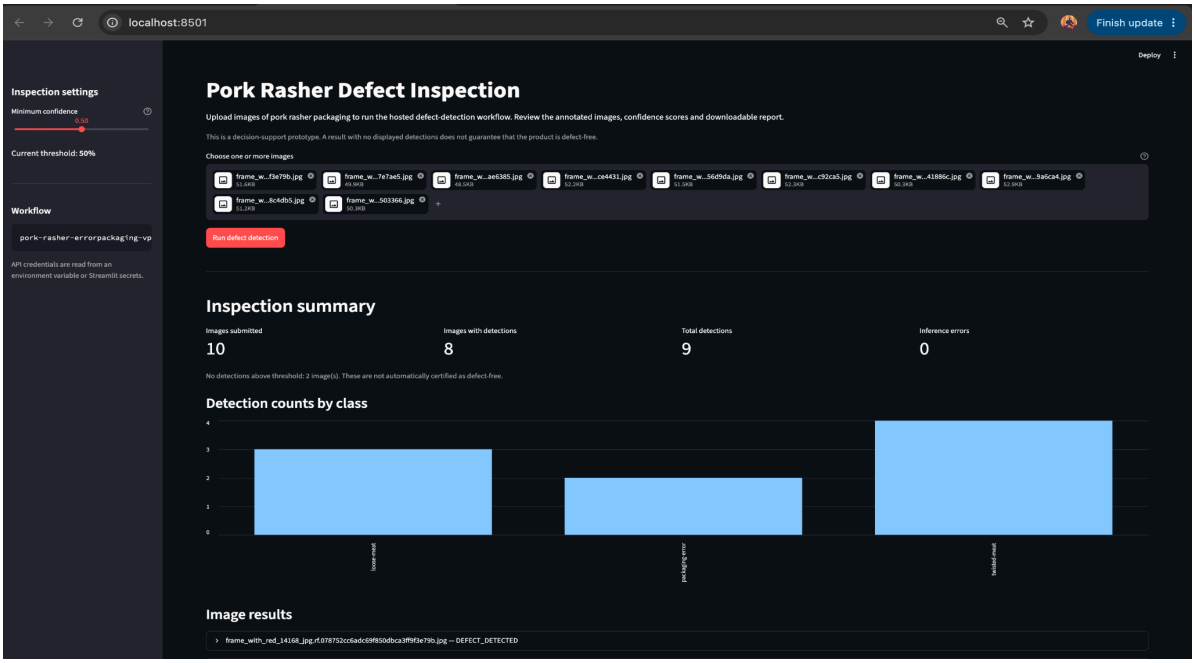


Figure 5: Screenshot of the application window processing test images

7.2 Example Application Input and Output

The application accepts a supported image file uploaded through the Streamlit interface. After the user initiates defect detection, the image is submitted to the hosted inference workflow. The application then displays the returned predictions, including defect labels, confidence scores and bounding boxes where available. The inspection results can be reviewed and exported using the available CSV and annotated-image ZIP download options

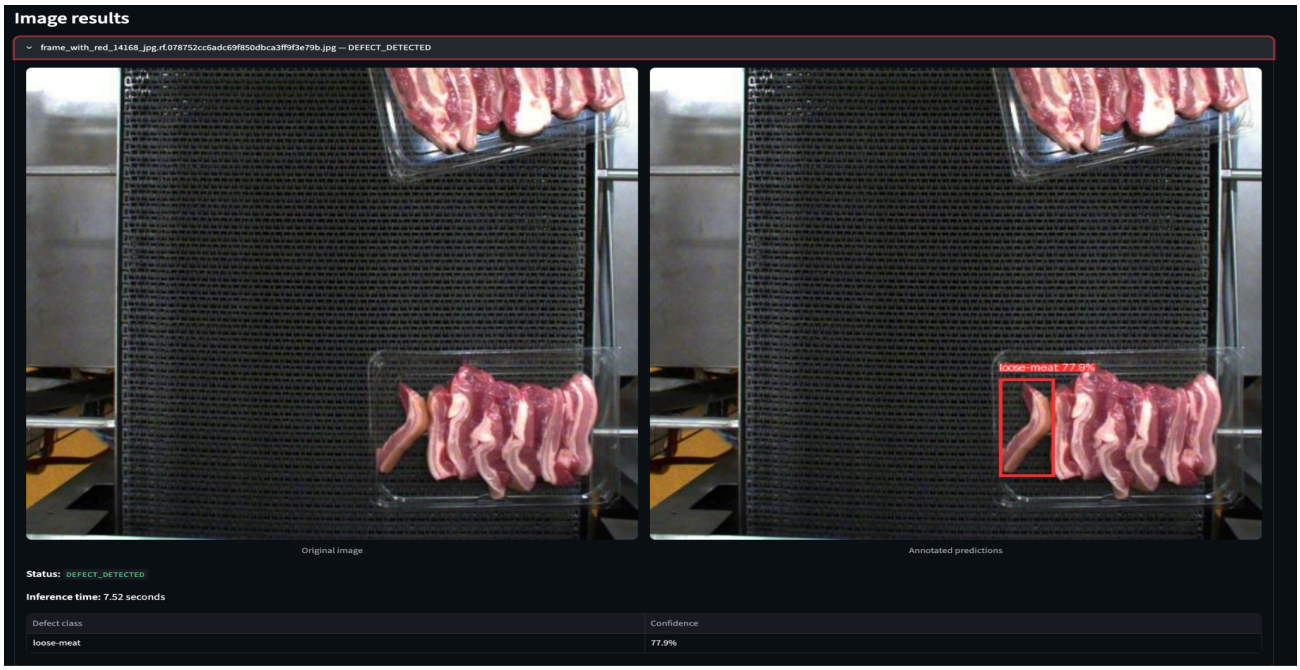


Figure 6**:** Streamlit interface showing test-image input, annotated detection output and result-export controls.

7.3 Repository and reproducibility

The GitHub repository contains the application and source code, README, dependency specification, meaningful comments/docstrings, tests, licence and generated HTML source documentation. Dataset versions, model settings and evaluation scripts are documented. API credentials, large private datasets and unnecessary generated outputs are excluded from version control. Recording software versions and using meaningful commits so the workflow can be reproduced.

8. Limitations and Engineering Discussion

- Class imbalance and low instance counts can make per-class results unstable, particularly for wrinkles.
- Annotation errors can corrupt training and make evaluation misleading; dataset review is required before final conclusions.
- Confidence thresholding changes the balance between false positives and false negatives.
- A no-detection output is not proof of a defect-free product.
- Hosted inference relies on internet connectivity, API access, workflow availability and service latency.
- Coordinate assumptions can misplace bounding boxes if preprocessing changes image dimensions.
- Image-only evaluation does not validate performance on a real production line with changing cameras, lighting, motion blur or throughput constraints.

From an engineering perspective, the key result is not simply whether the model produces bounding boxes, but whether it detects the defect types that matter at an acceptable false-alarm rate and processing time. The augmentation experiment illustrates that improving recall may reduce precision. The final system choice should therefore be based on the inspection cost of missed defects versus unnecessary manual reviews.

9. AI Use and Reflection

AI tools were used to support selected parts of the development and documentation process, including code organisation, explanation of errors, drafting of documentation and structuring of this report. Suggestions and generated content were reviewed against the implementation and observed results rather than assumed to be correct. During testing, attention was given to inference response fields, bounding-box coordinates, dependency compatibility and evaluation metrics. The final responsibility for the code, experimental design, interpretation of results and submitted report remains with the student. The main lesson from this process was that model outputs and evaluation metrics require independent verification, particularly when different evaluation pipelines are compared.

10. Conclusion and Future Work

This project developed a prototype computer-vision workflow for pork rasher packaging-defect inspection using a Roboflow-hosted RF-DETR inference service and YOLOv8n training experiments. The Streamlit interface supports image upload, annotated prediction review and CSV/ZIP export, while saved prediction records and metrics support repeatable evaluation.

The reported results show different strengths across the two approaches. RF-DETR achieved higher precision and F1-score in its custom evaluation, while the corrected-augmentation YOLOv8n evaluation reported higher recall and mAP@50. These results are provisional because the evaluation implementations must be aligned before a definitive model ranking can be made. The RF-DETR workflow's low recall for packaging-error, the imbalanced training distribution and the absence of wrinkle instances in the test split are important limitations.

Future work should verify and correct annotations, improve the representation of rare defect classes, evaluate both models on the same labelled test images using a common metric implementation, examine confidence-threshold sensitivity and conduct robustness tests under representative image conditions. Deployment on a production inspection line would require further validation of camera setup, lighting, throughput, latency and the consequences of missed defects. The current system should be treated as a decision-support prototype rather than a certified food-safety inspection system.

References

1. Monash University. (2026). *MMA3001 Numerical Methods and Machine Learning: Project brief*. Monash University.
2. Roboflow. (n.d.). *Inference SDK*. GitHub. https://github.com/roboflow/inference
3. Ultralytics. (n.d.). *Model training with Ultralytics YOLO*. https://docs.ultralytics.com/modes/train/
4. Ultralytics. (n.d.). *Model validation with Ultralytics YOLO*. https://docs.ultralytics.com/modes/val/
5. Ultralytics. (n.d.). *Model prediction with Ultralytics YOLO*. https://docs.ultralytics.com/modes/predict/
6. Roboflow. (n.d.). *RF-DETR: Real-time object detection and segmentation*. https://rfdetr.roboflow.com/
7. Roboflow Universe. (n.d.). *Pork Rasher Error Packaging dataset, Version 1*. Roboflow workspace name and workflow ID, Dataset
8. GitHub repo Link: https://github.com/Sameer462742/MMA_3001_Project.git

Appendix A — Application Testing and Reproducibility

B.1 Application workflow

The Streamlit application implements the following workflow:

1. Accept one or more supported image files.
2. Submit each image to the Roboflow-hosted inference workflow.
3. Retrieve predictions and filter them using the selected confidence threshold.
4. Display predicted defect labels, confidence scores and bounding boxes.
5. Summarise detections and inspection statuses.
6. Export results as a CSV report and an archive of annotated images.

Supported input formats are JPG, JPEG, PNG, BMP and WEBP. The default confidence threshold is 0.50.

B.2 Automated tests

The project's local test output previously recorded 10 tests passing with no failures. The test result should be retained as evidence only if it corresponds to the final submitted version of the application and test suite.

Test evidence	Recorded result
Automated tests executed	10
Passed	10
Failed	0

B.3 Software environment

The report records the following environment details:

Component	Recorded version or configuration
Operating system	macOS
Python	3.13.16
Roboflow Inference SDK	1.7.4
Streamlit	1.65.0
Inference service	Roboflow-hosted workflow
Local interface	Streamlit

B.4 Application limitations

The hosted inference workflow requires an internet connection and valid API access. Inference latency includes network and hosted-service overhead. Bounding-box drawing also depends on the prediction coordinates matching the dimensions of the original image; any resizing or cropping performed by the workflow must be checked.

A result with no detections above the selected confidence threshold does not prove that the image is free from defects. The application is a decision-support prototype and requires further validation before use on a production inspection line.